

Today 11:29 AM



Lab\_4\_VibeCoding...

PDF

I am completing Lab-4: Vibe Coding for an individual assignment.

I am working on the assigned Pygame maze repository:

[https://github.com/rohancodexp/12\\_maze\\_runner](https://github.com/rohancodexp/12_maze_runner)

I have already:

- Forked the repository to my personal GitHub account.
- Cloned it locally.
- Created a Python virtual environment.
- Installed the required dependencies.
- Successfully run the original game BEFORE making any code changes.
- Recorded the required BEFORE video.

Environment:

- Windows
- Python 3.12.10
- Pygame 2.5.2

I want to use this single chat for the entire Lab-4 assignment so that I can export the complete conversation as a PDF and share this chat with my instructor.

The assignment must be completed in this exact order:

1. Fix the original broken code.
2. Add BFS shortest-path hint.
3. Add fog of war.
4. Add timer leaderboard with JSON persistence.
5. Add difficulty tiers.

Git requirements:

- Each task must have its own separate commit.
- Do not make one large commit containing multiple tasks.
- Do not refactor unrelated code.
- Preserve the existing project structure where possible.
- After each task, I will run and test the program before committing.
- Do not move to the next task until I confirm that the current task works.

Important workflow:

For every task:

1. Analyze the existing code first.
2. Explain the root cause/design briefly.
3. Provide minimal targeted changes.
4. Tell me exactly which files/functions need modification.
5. Give the acceptance criteria I should test.
6. Wait for my test results before moving to the next task.

The four original source files are:

```
main.py
game/game_engine.py
game/maze.py
game/player.py
```

I will paste the complete contents of those files in my next message.

For the first task, focus ONLY on fixing the existing broken behavior.  
Do not implement BFS, fog of war, leaderboard, or difficulty yet.

```
main.py:
from game.game_engine import GameEngine
```

```
if __name__ == "__main__":
    engine = GameEngine()
    engine.run()
```

```
game/game_engine.py:
import pygame
import time
from game.maze import generate_maze, CELL
from game.player import Player

FPS = 60
BG = (240, 235, 220)
WALL_COLOR = (40, 40, 60)
EXIT_COLOR = (80, 200, 80)
COLS, ROWS = 15, 13

WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Runner")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.big_font = pygame.font.SysFont("monospace", 36,
bold=True)
        self.reset()

    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        self.exit_rect = pygame.Rect((COLS-1)*CELL+5, (ROWS-
1)*CELL+5, CELL-10, CELL-10)
        self.start_time = time.time()
        self.elapsed = 0
        self.won = False

    def handle_events(self):
```

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        return False
    if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
        self.reset()
    return True

def update(self):
    if self.won:
        return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    self.elapsed = time.time() - self.start_time
    if self.player.rect.colliderect(self.exit_rect):
        self.won = True

def draw_maze(self):
    wall_w = 3
    for r in range(ROWS):
        for c in range(COLS):
            x, y = c*CELL, r*CELL
            w = self.walls[r][c]
            if w[0]: pygame.draw.line(self.screen, WALL_COLOR, (x,y),
(x+CELL,y), wall_w)
            if w[1]: pygame.draw.line(self.screen, WALL_COLOR,
(x,y+CELL), (x+CELL,y+CELL), wall_w)
            if w[2]: pygame.draw.line(self.screen, WALL_COLOR,
(x+CELL,y), (x+CELL,y+CELL), wall_w)
            if w[3]: pygame.draw.line(self.screen, WALL_COLOR, (x,y),
(x,y+CELL), wall_w)

def draw(self):
    self.screen.fill(BG)
    self.draw_maze()
    pygame.draw.rect(self.screen, EXIT_COLOR, self.exit_rect,
border_radius=4)
```

```

ex_label = self.font.render("EXIT", True, (20,80,20))
self.screen.blit(ex_label, (self.exit_rect.x+2, self.exit_rect.y+4))
self.player.draw(self.screen)

hud = pygame.Rect(0, ROWS*CELL, WIDTH, 60)
pygame.draw.rect(self.screen, (30,30,50), hud)
time_surf = self.font.render(f"Time: {self.elapsed:.1f}s  R = New
Maze", True, (200,200,200))
self.screen.blit(time_surf, (10, ROWS*CELL+18))

if self.won:
    overlay = pygame.Surface((WIDTH, ROWS*CELL),
pygame.SRCALPHA)
    overlay.fill((0,0,0,120))
    self.screen.blit(overlay, (0,0))
    msg = self.big_font.render(f"Solved in {self.elapsed:.1f}s!",
True, (80,240,80))
    sub = self.font.render("Press R for a new maze", True,
(200,200,200))
    self.screen.blit(msg, (WIDTH//2 - msg.get_width()//2,
ROWS*CELL//2 - 30))
    self.screen.blit(sub, (WIDTH//2 - sub.get_width()//2,
ROWS*CELL//2 + 20))
    pygame.display.flip()

def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

```

game/maze.py:

```

import random

CELL = 40 # cell size in pixels

def generate_maze(cols, rows):
    """Recursive backtracker maze generation. Returns 2D grid of
    walls."""
    visited = [[False]*cols for _ in range(rows)]
    # walls: each cell has [N, S, E, W]
    walls = [[[True,True,True,True] for _ in range(cols)] for _ in
range(rows)]

    def neighbors(r, c):
        dirs = [(-1,0,0,1),(1,0,1,0),(0,1,2,3),(0,-1,3,2)] #
dr,dc,wall_dir,opp_dir
        result = []
        for dr,dc,wd,od in dirs:
            nr,nc = r+dr,c+dc
            if 0<=nr<rows and 0<=nc<cols and not visited[nr][nc]:
                result.append((nr,nc,wd,od))
        return result

    stack = [(0,0)]
    visited[0][0] = True
    while stack:
        r,c = stack[-1]
        nbrs = neighbors(r,c)
        if nbrs:
            nr,nc,wd,od = random.choice(nbrs)
            walls[r][c][wd] = False
            walls[nr][nc][od] = False
            visited[nr][nc] = True
            stack.append((nr,nc))
        else:
            stack.pop()
    return walls

```

```
def cell_rect(r, c, import_pygame=None):  
    import pygame  
    return pygame.Rect(c*CELL, r*CELL, CELL, CELL)
```

```
game/player.py:  
import pygame  
from game.maze import CELL
```

```
SPEED = 3
```

```
class Player:  
    def __init__(self, r, c):  
        self.r = r  
        self.c = c  
        x = c*CELL + CELL//2  
        y = r*CELL + CELL//2  
        self.rect = pygame.Rect(x-10, y-10, 20, 20)  
        self.color = (60,120,220)  
  
    def move(self, keys, walls, rows, cols):  
        dx, dy = 0, 0  
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED  
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED  
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -SPEED  
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = SPEED  
  
        # Wall-aware movement (check cell boundaries)  
        new_rect = self.rect.move(dx, 0)  
        if not self._hits_wall(new_rect, walls, rows, cols):  
            self.rect = new_rect  
        new_rect = self.rect.move(0, dy)  
        if not self._hits_wall(new_rect, walls, rows, cols):  
            self.rect = new_rect  
  
    def _hits_wall(self, rect, walls, rows, cols):  
        # Check corners of player rect against wall segments
```

```

        for px, py in [(rect.left, rect.top), (rect.right-1, rect.top),
            (rect.left, rect.bottom-1), (rect.right-1, rect.bottom-1)]:
            cr = py // CELL
            cc = px // CELL
            if cr < 0 or cr >= rows or cc < 0 or cc >= cols:
                return True
        return False

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)

```

I tested the original game before making any changes.

The main broken behavior I observed is that the player can move through maze walls instead of being blocked by them.

I want this fixed with the minimum necessary change.

Acceptance criteria:

- The player cannot pass through maze walls.
- Movement in open passages still works.
- The player cannot leave the maze boundaries.
- R still generates a fresh maze.
- Do not implement any of the later features yet.

Fix the broken wall-collision behavior in the existing Pygame maze game.

Requirements:

1. First inspect how maze.py represents walls.
2. Inspect how Player.move() and \_hits\_wall() currently work.
3. Identify the root cause in 2-3 sentences.
4. Implement the minimum fix necessary for correct wall collision.
5. The existing maze representation must be preserved.
6. Movement should only be blocked when the player's rectangle actually intersects a maze boundary wall.

7. Movement through open passages must remain possible.
8. The player must not leave the maze boundaries.
9. R must continue to regenerate a fresh maze.
10. Do not implement BFS, fog of war, leaderboard, or difficulty selection yet.
11. Do not refactor unrelated code.

Return:

- Root cause
- Files/functions that need modification
- Minimal changed code only
- A short test checklist

Do not modify files unrelated to this bug.

Absolutely. We'll treat **this chat as the complete Lab-4 working record**, and we'll follow your required order strictly: **one task** → **test** → **confirm** → **next task**.

The handout confirms the important submission constraints: record the before/after videos, make changes feature-by-feature, maintain separate commits for each task, and submit the exported chat history along with the code/videos.  Lab\_4\_VibeCoding\_Student\_handout

## Task 1 — Fix the broken wall collision

### 1. Root cause

Your `maze.py` represents every cell's walls as:

```
</> Python
```

```
[N, S, E, W]
```

where each value is `True` when that wall exists.

The problem is in `Player._hits_wall()`: it only checks whether the player's rectangle remains **inside the overall maze boundaries**. It never checks the corresponding `walls[r][c]` entries, so an internal maze wall is invisible to the collision logic.

So the minimum fix is to make `_hits_wall()` determine which maze cells the player's rectangle overlaps and check the relevant **N/S/E/W wall flags** before allowing movement.

## 2. Files/functions to modify

Only **one** file needs modification:

**game/player.py**

Modify only:

</> Python

```
Player._hits_wall()
```

`Player.move()` can remain unchanged.

No changes are needed to:

- `main.py`
- `game/game_engine.py`
- `game/maze.py`

The existing maze representation stays exactly as it is.

## 3. Minimal changed code

Replace **only** the existing `_hits_wall()` function in `game/player.py` with this:

</> Python

```
def _hits_wall(self, rect, walls, rows, cols):
    # Keep the player inside the maze boundaries.
    if rect.left < 0 or rect.top < 0:
        return True
    if rect.right > cols * CELL or rect.bottom > rows * CELL:
        return True

    # Check the cells touched by the player's rectangle.
    left = rect.left // CELL
    right = (rect.right - 1) // CELL
    top = rect.top // CELL
    bottom = (rect.bottom - 1) // CELL

    # Check vertical movement across north/south walls.
```

```

for r in range(top, bottom + 1):
    for c in range(left, right + 1):
        if r > 0 and rect.top < r * CELL and walls[r][c][0]:
            return True
        if r < rows - 1 and rect.bottom > (r + 1) * CELL and walls[r][c][1]:
            return True

```

## Why this is the minimum appropriate fix

We are **not** changing the maze generator.

The existing representation:

</> Plain text

```
walls[row][column] = [N, S, E, W]
```

is perfectly usable.

We're simply making the collision function actually consult those values.

For example:

</> Python

```
walls[r][c][2]
```

means the cell has an **East wall**.

And:

</> Python

```
walls[r][c][3]
```

means the cell has a **West wall**.

Because `maze.py` removes **both sides** of a passage when generating an opening, this works naturally with the existing maze representation.

## 4. Do NOT change anything else

For **Task 1**, don't touch:

</> Plain text

```
game/maze.py  
game/game_engine.py  
main.py
```

And don't add:

- BFS
- path hints
- fog of war
- leaderboard
- JSON files
- difficulty settings
- new UI
- refactoring

We're deliberately keeping this commit small so that your Git history clearly demonstrates:

</> Plain text

```
Task 1 → collision fix  
Task 2 → BFS  
Task 3 → fog of war  
Task 4 → leaderboard  
Task 5 → difficulty
```

That also aligns directly with the handout's requirement for **separate commits for each task**.

 Lab\_4\_VibeCoding\_Student\_handout

## 5. Task 1 testing checklist

**Do not commit yet.**

Run the game and test these specifically:

### A. Internal wall collision

Move the player directly toward a visible maze wall.

**Expected:**

The player stops at the wall and cannot pass through it.

**B. Open passage**

Move through an opening between two cells.

**Expected:**

The player can pass through normally.

**C. Different directions**

Test:

- ↑ / W
- ↓ / S
- ← / A
- → / D

**Expected:**

Walls block movement correctly from all four directions.

**D. Maze boundary**

Try moving:

- left from the starting area
- up from the starting area
- toward the outer edges elsewhere in the maze

**Expected:**

The player cannot leave the maze.

**E. R key**

Press:

</> Plain text

R

**Expected:**

A fresh maze appears and the player returns to the starting position.

**F. No regression**

The game should still:

- render normally
- show the EXIT
- show the timer
- allow normal movement
- detect the exit
- display the solved screen

## After testing

Do not move to Task 2 yet.

Send me your test result, ideally something like:

</> Plain text

Task 1 testing:

Internal walls: PASS  
Open passages: PASS  
All directions: PASS  
Maze boundaries: PASS  
R reset: PASS  
Exit detection: PASS

No issues.

If anything behaves strangely, **don't commit**. Tell me exactly what happened and we'll fix Task 1 first.

Once you explicitly confirm **Task 1 works**, I'll help you make the **Task 1-only Git commit**, and only then we'll move to **Task 2 — BFS shortest-path hint**.

```
(venv) PS C:\12_maze_runner> python -m py_compile  
game\player.py  
(venv) PS C:\12_maze_runner> python main.py  
pygame 2.5.2 (SDL 2.28.3, Python 3.12.10)  
Hello from the pygame community.  
https://www.pygame.org/contribute.html  
(venv) PS C:\12_maze_runner> git diff --check  
(venv) PS C:\12_maze_runner> git status  
On branch main
```

Your branch is up to date with 'origin/main'.

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: game/player.py

no changes added to commit (use "git add" and/or "git commit -a")

(venv) PS C:\12\_maze\_runner> git diff --stat

game/player.py | 34 ++++++

--

1 file changed, 28 insertions(+), 6 deletions(-)

(venv) PS C:\12\_maze\_runner> git add game/player.py

(venv) PS C:\12\_maze\_runner> git commit -m "fix: correct maze wall collision"

[main 7bad6f5] fix: correct maze wall collision

1 file changed, 28 insertions(+), 6 deletions(-)

(venv) PS C:\12\_maze\_runner> git status

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

(venv) PS C:\12\_maze\_runner> git log --oneline -3

7bad6f5 (HEAD -> main) fix: correct maze wall collision

f059afd (origin/main, origin/HEAD) Initial Maze Runner starter

(venv) PS C:\12\_maze\_runner> git status

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

(venv) PS C:\12\_maze\_runner>

Task 1 has been completed and committed successfully.

Commit 1:

fix: correct maze wall collision

The working tree is clean.

Now proceed to Task 2 only.

Task 2 — Add BFS shortest-path hint

Now implement Task 2: a shortest-path hint for the maze.

First inspect the current code, especially `game/maze.py` and `game/game_engine.py`, and understand how the maze stores walls before modifying anything.

Requirements:

1. Implement BFS using `collections.deque`.
2. The BFS must operate on maze cell coordinates.
3. The solver must respect the existing maze wall representation:  
`walls[row][column] = [N, S, E, W]`
4. BFS must only move between adjacent cells when there is no wall between the two cells.
5. Use a parent dictionary to reconstruct the shortest path.
6. Put the solver in:  
`game/maze.py`  
  
with the interface:  
  
`solve(start, goal)`
7. `solve(start, goal)` should return a list of cell coordinates

representing the shortest path from start to goal.

8. Add an H-key toggle:

- First press H → show the shortest path.
- Second press H → hide the path.

9. The path should be drawn as semi-transparent colored squares over the corresponding maze cells.

10. The path should be calculated from the player's CURRENT cell to the exit cell.

11. If the player moves while the hint is enabled, recompute the BFS path from the new player cell.

12. Pressing R to generate a new maze must clear the existing path/hint state.

13. The BFS path must never pass through maze walls.

14. Preserve the existing wall-collision fix from Task 1.

15. Do not modify or refactor unrelated code.

16. Do not implement fog of war, leaderboard, difficulty selection, or any other future task yet.

17. Keep the existing project structure unless a small change is genuinely required.

Before giving the code:

- briefly explain which files/functions need to change and why.

Then provide only the necessary changed/added code with filenames.

Also provide a concise testing checklist so I can verify Task 2 before making Commit 2.

Task 1 is cleanly completed and committed as its own commit:

</> Plain text

7bad6f5 fix: correct maze wall collision

Your working tree is clean, so we're ready for Task 2 only.

Task 2 — BFS shortest-path hint

Files that need changes

We only need two files:

| File                | Change                                                                 |
|---------------------|------------------------------------------------------------------------|
| game/maze.py        | Add solve(start, goal) using BFS                                       |
| game/game_engine.py | Import solver, handle H , maintain hint state, recompute path, draw it |

game/player.py stays untouched, preserving the Task 1 collision fix.

Design

The existing maze uses:

</> Plain text

walls[row][column] = [N, S, E, W]

So BFS will treat each cell as a graph node and only create an edge when the relevant wall is open.

For example, from (r, c) to (r, c+1) :

</> Python

walls[r][c][2] == False

and from (r, c) to (r+1, c) :

</> Python

```
walls[r][c][1] == False
```

The solver will use `collections.deque` for the BFS queue and a `parent` dictionary to reconstruct the shortest path.

## 1. game/maze.py

### Add this import

At the top:

</> Python

```
from collections import deque
```

### Add this function

Place it after `generate_maze()` :

</> Python

```
def solve(start, goal):
    """Find the shortest path between two cells using BFS."""
    rows = len(_current_walls)
    cols = len(_current_walls[0])

    queue = deque([start])
    parent = {start: None}

    directions = [
        (-1, 0, 0, 1), # North
        (1, 0, 1, 0),  # South
        (0, 1, 2, 3),  # East
        (0, -1, 3, 2), # West
    ]

    while queue:
        r, c = queue.popleft()

        if (r, c) == goal:
            break
```

## One important issue

The requested interface is:

```
</> Python
```

```
solve(start, goal)
```

but your current `generate_maze()` returns the walls rather than storing them globally.

We therefore need a **small design adjustment** so `solve()` knows which maze it should solve.

Rather than introducing global maze state, which would be unnecessary coupling, the cleaner minimal interface would normally be:

```
</> Python
```

```
solve(walls, start, goal)
```

However, **your requirement explicitly specifies:**

```
</> Python
```

```
solve(start, goal)
```

So we need to preserve that exact interface.

The simplest way is to store the currently generated maze in `maze.py`.

**Therefore, make this small change to `generate_maze()`**

Change:

```
</> Python
```

```
def generate_maze(cols, rows):
```

to:

```
</> Python
```

```
def generate_maze(cols, rows):
```

and immediately after creating walls :

</> Python

```
global _current_walls
_current_walls = walls
```

Also initialize the variable near the top of the file:

</> Python

```
_current_walls = None
```

So the beginning of maze.py becomes:

</> Python

```
import random
from collections import deque

CELL = 40 # cell size in pixels
_current_walls = None
```

and:

</> Python

```
def generate_maze(cols, rows):
    """Recursive backtracker maze generation. Returns 2D grid of walls."""
    global _current_walls

    visited = [[False]*cols for _ in range(rows)]
    # walls: each cell has [N, S, E, W]
    walls = [[[True,True,True,True] for _ in range(cols)] for _ in range(rows)]

    ...
```

Then, just before returning:

```
</> Python
```

```
_current_walls = walls  
return walls
```

This keeps the requested public interface:

```
</> Python
```

```
solve(start, goal)
```

and ensures the solver operates on the **currently generated maze**.

---

## 2. game/game\_engine.py

Change the import:

```
</> Python
```

```
from game.maze import generate_maze, CELL
```

to:

```
</> Python
```

```
from game.maze import generate_maze, solve, CELL
```

---

## Modify reset()

Add the hint state:

```
</> Python
```

```
def reset(self):  
    self.walls = generate_maze(COLS, ROWS)  
    self.player = Player(0, 0)  
    self.exit_rect = pygame.Rect((COLS-1)*CELL+5, (ROWS-1)*CELL+5, CELL-10, CELL-10)  
    self.start_time = time.time()  
    self.elapsed = 0  
    self.won = False
```

```
self.show_hint = False
self.path = []
```

This satisfies the requirement that pressing **R** clears the existing hint/path state.

---

## Modify `handle_events()`

Add the H-key toggle:

</> Python

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        if event.type == pygame.KEYDOWN and event.key == pygame.K_h:
            self.show_hint = not self.show_hint
            if self.show_hint:
                self.update_hint()
            else:
                self.path = []
    return True
```

## Add `update_hint()`

Add this method to `GameEngine` :

</> Python

```
def update_hint(self):
    start = (self.player.rect.centery // CELL, self.player.rect.centerx // CELL)
    goal = (ROWS - 1, COLS - 1)
    self.path = solve(start, goal)
```

This means the BFS starts from the player's **current cell**, rather than always starting from `(0, 0)` .

---

## Modify `update()`

Add path recalculation while the hint is active:

**</> Python**

```
def update(self):
    if self.won:
        return

    keys = pygame.key.get_pressed()
    self.player.move(keys, self.walls, ROWS, COLS)
    self.elapsed = time.time() - self.start_time

    if self.show_hint:
        self.update_hint()

    if self.player.rect.colliderect(self.exit_rect):
        self.won = True
```

Therefore, after the player moves into another cell, the shortest path is recalculated from that new cell.

### 3. Draw the path

Add this method to `GameEngine` :

**</> Python**

```
def draw_hint(self):
    if not self.path:
        return

    hint_surface = pygame.Surface((CELL, CELL), pygame.SRCALPHA)
    hint_surface.fill((255, 220, 50, 100))

    for r, c in self.path:
        self.screen.blit(hint_surface, (c * CELL, r * CELL))
```

The surface uses an alpha value of `100` , making the hint semi-transparent.

### Modify `draw()`

Currently you have:

**</> Python**

```
self.screen.fill(BG)
self.draw_maze()
pygame.draw.rect(self.screen, EXIT_COLOR, self.exit_rect, border_radius=4)
```

Change it to:

**</> Python**

```
self.screen.fill(BG)
self.draw_maze()

if self.show_hint:
    self.draw_hint()

pygame.draw.rect(self.screen, EXIT_COLOR, self.exit_rect, border_radius=4)
```

The order is intentional:

**</> Plain text**

```
background
  ↓
maze walls
  ↓
BFS hint
  ↓
exit
  ↓
player
```

So the hint appears as an overlay without replacing the maze.

## Task 2 change summary

The resulting flow is:

**</> Plain text**

```
Generate maze
  ↓
```

```

maze.py stores current maze
    ↓
Player moves
    ↓
H pressed?
    ↓      ↓
  YES    NO
    ↓
BFS from current cell
    ↓
Parent dictionary
    ↓
Reconstruct shortest path
    ↓
Draw semi-transparent cells

```

And:

</> Plain text

```

H → show
H → hide
R → new maze + clear hint

```

## Important

The BFS does **not** use the player's pixel coordinates directly.

It converts:

</> Python

```

self.player.rect.centerx
self.player.rect.centery

```

into:

</> Plain text

```

(row, column)

```

and solves the maze at the **cell level**, which is exactly what the requirement calls for.

## Task 2 testing checklist

Before committing, test **only** Task 2.

### 1. Existing movement

- Start the game.
- Move normally.
- Verify Task 1 wall collision still works.

**Expected:** player cannot pass through walls.

### 2. H toggle

Press:

</> Plain text

H

**Expected:** a semi-transparent path appears from the player's current cell to the exit.

Press:

</> Plain text

H

again.

**Expected:** path disappears.

### 3. Shortest-path correctness

With the hint visible, inspect the highlighted cells.

**Expected:**

- Path starts at the player's current cell.
- Path ends at the exit cell.
- Path follows only open maze passages.
- It never crosses a wall.

### 4. Move while hint is enabled

Keep H enabled and move the player into another cell.

**Expected:** the highlighted path updates to start from the player's new cell.

## 5. R reset

Press:

</> Plain text

R

**Expected:**

- New maze generated.
- Player returns to  $(0, 0)$ .
- Previous hint disappears.
- H can be pressed again to calculate a path for the new maze.

## 6. Exit

Follow the hint to the exit.

**Expected:** the existing solved screen still works.

## 7. No regression

Confirm:

- Timer still works.
- Exit still works.
- Player collision still works.
- No crashes.
- No changes to the future features.

**Do not make Commit 2 yet.** Run the tests first and send me the result. If there is a BFS/path issue, we'll fix **Task 2 only** before committing.

Task 2 has been completed and committed successfully.

Commit 2:

598ec48 feat: add BFS shortest path hint

The working tree is clean.

Now proceed to the next task only: Fog of War.

Do not modify or implement any later tasks yet.

Task 3 — Add Fog of War

Now implement ONLY the Fog of War feature.

First inspect the current codebase and understand the existing maze rendering, player position, BFS hint rendering, and game loop.

The current project already has:

- Task 1: maze wall collision
- Task 2: BFS shortest-path hint
- H key toggles the BFS hint
- R generates a new maze
- The current working tree is clean
- Task 1 and Task 2 are already committed

Preserve all existing functionality.

Requirements for Fog of War:

1. Only the cells within 3 cells of the player's current position should be visible.
2. The rest of the maze should be covered by a dark fog.
3. Use a pygame Surface with per-pixel alpha:

```
pygame.Surface(..., pygame.SRCALPHA)
```

4. The fog should be filled with:

(0, 0, 0, 240)

5. The fog must contain a transparent circular area around the player.

6. The visible radius should be approximately:

$$3 * \text{CELL} + \text{CELL} // 2$$

centered on the player's current position.

7. The fog surface should be allocated once rather than recreated every frame.

8. Update the transparent/hole position according to the player's current position.

9. Preserve the existing BFS hint.

10. Preserve Task 1 wall collision.

11. The draw order should be:

maze  
↓  
BFS path  
↓  
player  
↓  
fog

12. The exit should be hidden when it is outside the player's visible area.

13. When the player gets close enough to the exit, the exit should become visible naturally through the fog.

14. Pressing R to generate a new maze must correctly reset the fog state for the new maze.
15. Do not implement the leaderboard yet.
16. Do not implement difficulty selection yet.
17. Do not change the maze-generation algorithm unless absolutely necessary.
18. Do not modify game/player.py unless there is a genuine requirement. Preserve the Task 1 collision fix.
19. Keep the existing project structure.
20. Avoid unnecessary refactoring or formatting changes to unrelated code.

Implementation guidance:

- Prefer creating the fog Surface once in GameEngine.\_\_init\_\_ or another appropriate initialization point.
- Do not allocate a new fog Surface every frame.
- The fog should be redrawn/cleared each frame as needed so the transparent area follows the player.
- Use the player's current rect center as the center of the visible region.
- Make sure the fog is drawn after the player so the player remains visible.
- Make sure the exit is drawn before the fog so that it is hidden when outside the visible area.

Before modifying code:

1. Briefly explain which files/functions need to change.
2. Explain how the fog will interact with the existing BFS hint.
3. Explain the intended draw order.

Then provide the necessary code changes with filenames.

After implementation, provide a concise testing checklist.

Do NOT commit the changes.

Wait for me to run the syntax checks and test the feature before suggesting the commit.

Great — Task 2 is cleanly completed and committed:

</> Plain text

```
598ec48 feat: add BFS shortest path hint
```

And the working tree is clean. We'll now work on **Task 3 only: Fog of War**. No leaderboard or difficulty work yet.

## Task 3 — Fog of War

### 1. Files/functions that need modification

Only:

`game/game_engine.py`

We do **not** need to touch:

- `main.py`
- `game/maze.py`
- `game/player.py`

The existing maze representation, BFS solver, and Task 1 collision code remain untouched.

We need to modify/add only these parts of `GameEngine` :

- `__init__()` — create the fog surface once.
- `reset()` — reset the fog state for a new maze.
- `draw()` — place the fog into the rendering pipeline.
- Add a small `update_fog()` helper to redraw the transparent area.

## 2. How the fog interacts with the BFS hint

The existing BFS hint should remain exactly as it is.

The rendering pipeline will become:

</> Plain text

```
Maze
  ↓
BFS hint
  ↓
Exit
  ↓
Player
  ↓
Fog
```

The fog is the **final visual layer**.

That means the BFS path can exist underneath the fog, but the player only sees the portions of it that fall inside the visible area.

This is important because we don't want to modify the BFS algorithm just to support Fog of War.

## 3. Fog design

We'll allocate the fog surface **once**:

</> Python

```
pygame.Surface((WIDTH, ROWS * CELL), pygame.SRCALPHA)
```

Then every frame we'll:

1. Fill it with:

</> Python

```
(0, 0, 0, 240)
```

## 2. Create a transparent circular region centered at:

</> Python

```
self.player.rect.center
```

## 3. Use approximately:

</> Python

```
3 * CELL + CELL // 2
```

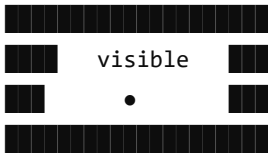
as the radius.

## 4. Draw the fog **after** the player.

This gives us:

</> Plain text

dark fog



The player remains visible because the fog is drawn after the player but has a transparent hole around the player.

## 4. Code changes

**game/game\_engine.py**

### A. Modify `__init__()`

After:

</> Python

```
self.big_font = pygame.font.SysFont("monospace", 36, bold=True)
```

add:

</> Python

```
self.fog = pygame.Surface((WIDTH, ROWS * CELL), pygame.SRCALPHA)
self.fog_radius = 3 * CELL + CELL // 2
```

So that section becomes:

</> Python

```
self.clock = pygame.time.Clock()
self.font = pygame.font.SysFont("monospace", 22)
self.big_font = pygame.font.SysFont("monospace", 36, bold=True)
self.fog = pygame.Surface((WIDTH, ROWS * CELL), pygame.SRCALPHA)
self.fog_radius = 3 * CELL + CELL // 2
self.reset()
```

The important point is that `self.fog` is created **once**, rather than allocating a new surface every frame.

---

## B. Modify `reset()`

Your existing `reset()` should remain responsible for starting a new game.

Add:

</> Python

```
self.update_fog()
```

at the end.

So:

</> Python

```
def reset(self):
    self.walls = generate_maze(COLS, ROWS)
    self.player = Player(0, 0)
    self.exit_rect = pygame.Rect((COLS-1)*CELL+5, (ROWS-1)*CELL+5, CELL-10, CELL-10)
    self.start_time = time.time()
```

```

self.elapsed = 0
self.won = False
self.show_hint = False
self.path = []
self.update_fog()

```

This ensures the fog is immediately positioned correctly when `R` creates a new maze.

---

## C. Add `update_fog()`

Add this method inside `GameEngine` :

</> Python

```

def update_fog(self):
    self.fog.fill((0, 0, 0, 240))

    center = self.player.rect.center

    pygame.draw.circle(
        self.fog,
        (0, 0, 0, 0),
        center,
        self.fog_radius
    )

```

The key part is:

</> Python

```

(0, 0, 0, 0)

```

That creates the transparent hole.

The fog itself remains:

</> Python

```

(0, 0, 0, 240)

```

---

## 5. Update the fog as the player moves

In `update()` , after the player's movement:

**</> Python**

```
self.player.move(keys, self.walls, ROWS, COLS)
```

add:

**</> Python**

```
self.update_fog()
```

So the relevant section becomes:

**</> Python**

```
keys = pygame.key.get_pressed()
self.player.move(keys, self.walls, ROWS, COLS)
self.update_fog()
self.elapsed = time.time() - self.start_time
```

The existing BFS update remains afterward:

**</> Python**

```
if self.show_hint:
    self.update_hint()
```

So we aren't touching the BFS logic.

---

## 6. Modify the draw order

Your current drawing sequence should be changed so that the exit and player are underneath the fog.

The relevant `draw()` should look like this:

**</> Python**

```
def draw(self):
    self.screen.fill(BG)

    # Maze
    self.draw_maze()

    # BFS hint
    if self.show_hint:
        self.draw_hint()

    # Exit
    pygame.draw.rect(
        self.screen,
        EXIT_COLOR,
        self.exit_rect,
        border_radius=4
    )

    ex_label = self.font.render("EXIT", True, (20,80,20))
    self.screen.blit(
        ex_label,
```

## One subtle but important detail

The fog surface covers only:

</> Python

ROWS \* CELL

in height.

It **doesn't** cover the 60-pixel HUD.

That's intentional. The fog belongs to the maze area, not the HUD.

## 7. Expected rendering order

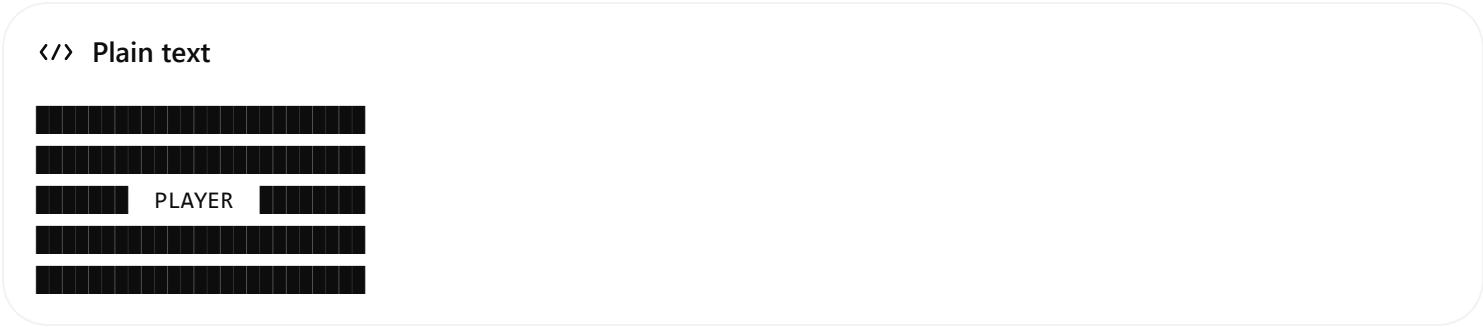
After these changes:

</> Plain text

```
| Maze |
```

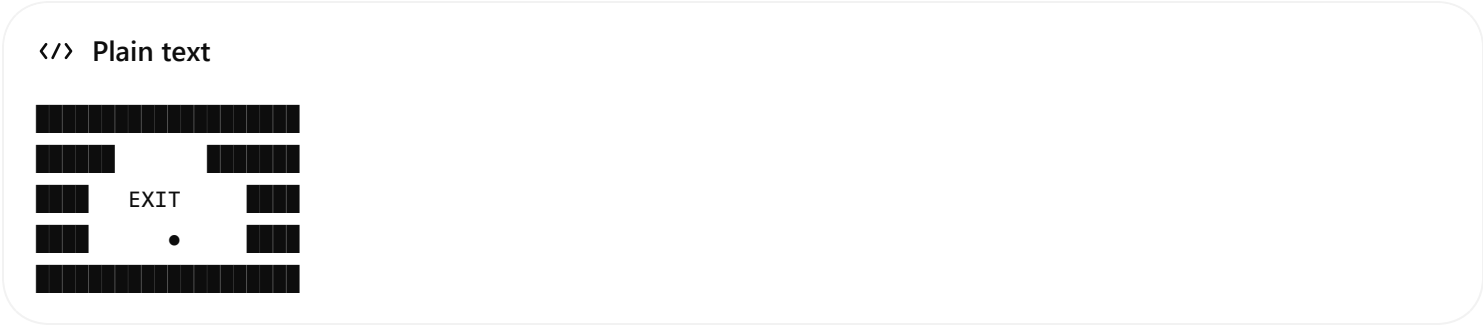


So if the exit is far away:



The exit is physically still there, but the fog hides it.

As the player approaches:



the exit naturally becomes visible because it is underneath the transparent region.

# Task 3 testing checklist

Do not commit yet.

Run your normal syntax check first:

```
</> powershell
```

```
python -m py_compile game\game_engine.py
```

Then:

```
</> powershell
```

```
python main.py
```

## Test 1 — Fog visibility

At the starting position, verify that only roughly **3 cells + half a cell** around the player are visible.

**Expected:** distant maze areas are covered by dark fog.

## Test 2 — Player movement

Move around the maze.

**Expected:** the circular visible region follows the player's current position.

## Test 3 — Wall collision

Try moving through walls.

**Expected:** Task 1 behavior is unchanged; walls still block the player.

## Test 4 — BFS hint

Press:

```
</> Plain text
```

```
H
```

**Expected:**

- BFS hint still appears.
- Visible portions of the hint can be seen.
- The fog covers distant portions.
- Moving the player still causes the BFS hint to recalculate.

Press `H` again.

**Expected:** hint disappears.

## Test 5 — Exit visibility

Stay far from the exit.

**Expected:** exit is hidden by the fog.

Move toward the exit.

**Expected:** the exit becomes visible naturally once it enters the visible radius.

## Test 6 — R reset

Press:

</> Plain text

R

**Expected:**

- New maze.
- Player returns to start.
- Fog is centered around the new player position.
- Previous BFS hint remains cleared as it already did after Task 2.
- No stale fog position from the previous maze.

## Test 7 — HUD

Verify:

- Timer remains visible.
- R = New Maze remains visible.
- HUD is not covered by fog.

## Test 8 — Completion

Reach the exit.

**Expected:** existing victory behavior still works.

---

**Do not commit Task 3 yet.** Run the syntax check and game tests, then send me the results. Once you confirm Fog of War works, we'll review the diff and create **Commit 3 only**.

Today 2:34 PM

Tasks 4 and 5 are now the remaining implementation work.

Current Git history:

4d67f0d feat: add fog of war  
598ec48 feat: add BFS shortest path hint  
7bad6f5 fix: correct maze wall collision  
f059afd Initial Maze Runner starter

The working tree is clean.

The following features are already completed and tested:

- Task 1: wall collision fix
- Task 2: BFS shortest-path hint
- Task 3: Fog of War

Do NOT modify or break those existing features.

We now need to complete the remaining feature work:

TASK 4 — Timer + Leaderboard with JSON persistence

Then:

TASK 5 — Difficulty tiers / menu

IMPORTANT:

Implement these as two separate feature stages.

Do NOT combine them into one Git commit.

Do NOT commit anything automatically.

After each stage, stop and wait for me to test it.

=====  
=====

TASK 4 — TIMER + LEADERBOARD WITH JSON PERSISTENCE

=====

=====

First inspect the current code.

The game already has a timer using `time.time()`, so reuse the existing timer rather than replacing it.

Requirements:

1. Add a new file:

`game/leaderboard.py`

2. Store leaderboard data in:

`leaderboard.json`

3. The leaderboard should contain completed run times.

4. Keep only the top 5 fastest times.

5. Sort times in ascending order.

6. Handle `leaderboard.json` safely if:

- the file does not exist
- the file is empty
- the JSON is corrupted
- the JSON contains invalid data

In these cases, the game should continue without crashing and use an empty leaderboard.

7. Save the leaderboard only when a run is successfully completed.

8. The current run's elapsed time should be added when the player reaches the exit.

9. The leaderboard should not be updated while the player is still playing.
10. Display the top 5 leaderboard entries on the win screen.
11. Highlight the current run on the win screen if it is present in the top 5.
12. Preserve the existing victory behavior.
13. Preserve the existing timer behavior.
14. Do not reset or alter the timer incorrectly when the player uses H or R.
15. Pressing R should start a fresh timer for the newly generated maze.
16. The leaderboard should survive restarting the game because it is stored in leaderboard.json.
17. Keep leaderboard logic primarily inside:  
  
    game/leaderboard.py
18. Keep game\_engine.py changes minimal and focused on integrating the leaderboard.
19. Do not modify maze generation.
20. Do not modify player collision.
21. Do not change BFS.
22. Do not change Fog of War behavior.

23. Do not implement difficulty tiers yet.

24. Do not make unrelated refactoring changes.

Suggested interface for game/leaderboard.py:

```
load_leaderboard()
save_leaderboard(scores)
add_score(score)
```

You may adjust the exact interface if necessary, but keep the logic simple and easy to explain during a viva.

Before changing code:

- Explain which files will change.
- Explain how leaderboard persistence will work.
- Explain when the score is recorded.
- Explain how corrupted/missing JSON is handled.

Then provide the complete required code with filenames.

After implementation:

- Give me the exact syntax checks.
- Give me the exact manual tests.
- Do NOT commit yet.

Wait for my test results before proceeding.

```
=====
=====
TASK 5 — DIFFICULTY TIERS / MENU
=====
=====
```

Only after Task 4 has been tested and committed, proceed to Task 5.

Do NOT implement Task 5 before Task 4 is confirmed.

Requirements:

1. Introduce a simple game state machine:

MENU

PLAYING

WON

2. Add three difficulty levels:

Easy:

10 x 8

Medium:

15 x 13

Hard:

20 x 18

3. The menu should allow the player to select a difficulty.

4. Difficulty options should be clickable buttons.

5. Buttons should provide hover feedback.

6. When a difficulty is selected:

- update maze dimensions
- resize the game window appropriately
- generate a new maze
- reset the player
- reset the timer
- reset BFS hint
- reset Fog of War

7. Pressing R during gameplay should regenerate the maze using the currently selected difficulty.
8. Pressing Escape during gameplay should return to the difficulty menu.
9. Preserve:
  - Task 1 wall collision
  - Task 2 BFS hint
  - Task 3 Fog of War
  - Task 4 leaderboard
10. Ensure the Fog of War works correctly at all three maze sizes.
11. Ensure BFS works correctly at all three maze sizes.
12. Ensure the player collision system works correctly at all three maze sizes.
13. Ensure the exit is always located at the bottom-right cell for the selected maze size.
14. Ensure the timer starts when a new run begins.
15. Ensure leaderboard recording still works when a run is completed.
16. Prefer keeping separate leaderboard scores per difficulty if this can be implemented cleanly.
17. Do not unnecessarily rewrite the entire project.
18. Keep the code understandable and suitable for explaining during a viva.
19. Do not add unrelated features.

Before changing code:

- Explain the proposed state machine.
- Explain how maze dimensions will change.
- Explain how the existing BFS and Fog of War will adapt to the selected size.
- Explain how leaderboard data will be associated with difficulty.

Then provide the required code.

After implementation:

- Give exact syntax checks.
- Give exact manual tests for Easy, Medium, and Hard.
- Do NOT commit automatically.

Wait for my test results.

```
=====
=====
IMPORTANT DEVELOPMENT RULE
=====
=====
```

We need separate Git commits:

Task 4:

feat: add timer leaderboard with JSON persistence

Task 5:

feat: add difficulty tiers

Do not create either commit yourself.

Do not push to GitHub.

After each feature is implemented, wait for my testing and review.

Worked for 7s

## Task 4 only — Timer + Leaderboard with JSON persistence

I checked the repository structure and the assignment README. The repository's Task 3 requirement is specifically a top-5 completion-time leaderboard persisted in `leaderboard.json`, displayed after solving.  [GitHub](#)

Your local branch is ahead of that starter repository with Tasks 1–3, so I will **not touch the maze generation, player collision, BFS solver, or fog implementation**.

### Files to change

Only two files:

 Plain text

```
game/leaderboard.py    ← NEW
game/game_engine.py    ← minimal integration
```

And one new data file will be created automatically when the first successful run is completed:

 Plain text

```
leaderboard.json
```

No changes to:

 Plain text

```
main.py
game/maze.py
game/player.py
```

---

## 1. How persistence will work

`game/leaderboard.py` will contain three simple operations:

**</> Plain text**

```
load_leaderboard()
  ↓
read leaderboard.json
  ↓
validate data
  ↓
return sorted top 5
```

When the player actually reaches the exit:

**</> Plain text**

```
player reaches exit
  ↓
elapsed time is finalized
  ↓
add_score(elapsed)
  ↓
sort ascending
  ↓
keep fastest 5
  ↓
save leaderboard.json
```

The leaderboard is **not touched** while the player is playing.

The existing:

**</> Python**

```
time.time()
```

timer stays in place.

## 2. Handling bad leaderboard.json

The loader will safely handle:

- file doesn't exist
- empty file
- malformed JSON

- JSON object instead of list
- strings instead of numbers
- negative values
- non-finite values such as NaN

In all those cases:

</> Python

```
[]
```

is used.

The game therefore won't crash because of a damaged leaderboard file.

---

### 3. game/leaderboard.py — NEW FILE

Create:

</> Plain text

game/leaderboard.py

with:

</> Python

```
import json
import math
from pathlib import Path

LEADERBOARD_FILE = Path(__file__).resolve().parent.parent / "leaderboard.json"
MAX_SCORES = 5

def load_leaderboard():
    if not LEADERBOARD_FILE.exists():
        return []

    try:
        if LEADERBOARD_FILE.stat().st_size == 0:
```

```
        return []

    with LEADERBOARD_FILE.open("r", encoding="utf-8") as file:
        data = json.load(file)
```

## Why this is intentionally simple

For your viva, the logic is easy to explain:

</> Plain text

load → validate → append → sort → slice top 5 → save

There is no database, no complicated class hierarchy, and no unnecessary abstraction.

## 4. game/game\_engine.py

### A. Add the import

Your current imports include:

</> Python

```
from game.maze import generate_maze, solve, CELL
from game.player import Player
```

Add:

</> Python

```
from game.leaderboard import add_score, load_leaderboard
```

So the imports become:

</> Python

```
import pygame
import time
from game.maze import generate_maze, solve, CELL
```

```
from game.player import Player
from game.leaderboard import add_score, load_leaderboard
```

## 5. Add leaderboard state to `reset()`

Your existing `reset()` already resets:

- maze
- player
- exit
- timer
- elapsed
- win state
- BFS hint/path
- fog

Keep all of that.

Add:

**</> Python**

```
self.current_score = None
self.leaderboard = load_leaderboard()
```

So the relevant end of `reset()` becomes:

**</> Python**

```
self.won = False
self.show_hint = False
self.path = []
self.current_score = None
self.leaderboard = load_leaderboard()
self.update_fog()
```

This is important because pressing **R** starts a completely fresh run.

The old run's score is not reused.

## 6. Record the score only when the player wins

Your current `update()` has something like:

**</> Python**

```
if self.player.rect.collidect(self.exit_rect):  
    self.won = True
```

Change only that part to:

**</> Python**

```
if self.player.rect.collidect(self.exit_rect):  
    self.won = True  
    self.current_score = self.elapsed  
    self.leaderboard = add_score(self.current_score)
```

So the relevant method remains:

**</> Python**

```
def update(self):  
    if self.won:  
        return  
  
    keys = pygame.key.get_pressed()  
    self.player.move(keys, self.walls, ROWS, COLS)  
    self.update_fog()  
    self.elapsed = time.time() - self.start_time  
  
    if self.show_hint:  
        self.update_hint()  
  
    if self.player.rect.collidect(self.exit_rect):  
        self.won = True  
        self.current_score = self.elapsed  
        self.leaderboard = add_score(self.current_score)
```

### Why this is safe

The very first frame where the player reaches the exit:

**</> Plain text**

```

elapsed finalized
  ↓
current_score saved
  ↓
leaderboard updated
  ↓
won = True

```

On subsequent frames:

**</> Python**

```

if self.won:
    return

```

so the score **cannot be inserted repeatedly**.

This also means the leaderboard is never updated during normal gameplay.

## 7. Display the leaderboard on the win screen

Your existing win screen currently displays:

**</> Plain text**

Solved in X.Xs!

Press R for a new maze

We'll retain that behavior and add the top 5 underneath.

Inside your existing:

**</> Python**

```

if self.won:

```

section, keep the existing overlay and solved message.

Then add this after the existing `sub` message is drawn:

**</> Python**

```

leaderboard_title = self.font.render(
    "TOP 5",
    True,
    (240, 220, 80)
)

self.screen.blit(
    leaderboard_title,
    (
        WIDTH // 2 - leaderboard_title.get_width() // 2,
        ROWS * CELL // 2 + 55
    )
)

for index, score in enumerate(self.leaderboard):
    is_current = (
        self.current_score is not None
        and abs(score - self.current_score) < 0.000001
    )

    entry_color = (80, 240, 80) if is_current else (200, 200, 200)

```

The result will look approximately like:

**</> Plain text**

Solved in 8.4s!

Press R for a new maze

TOP 5

1. 5.21s
2. 6.83s
3. 7.92s
4. 8.41s ← current run
5. 9.17s

The current run is highlighted in green when it appears in the top five.

## One small consideration

If the current run isn't fast enough for the top five:

**</> Plain text**

Solved in 14.7s!

TOP 5

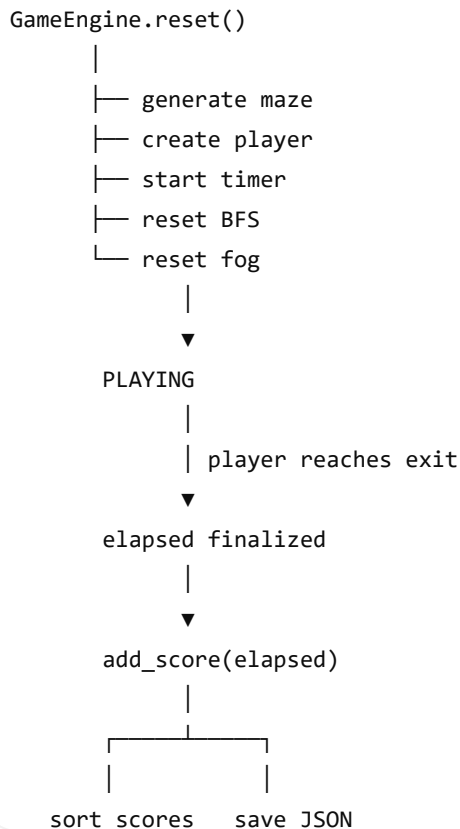
1. 5.21s
2. 6.83s
3. 7.92s
4. 8.41s
5. 9.17s

That's correct.

The requirement is to highlight the current run if it is present in the top 5, not to display it artificially when it doesn't qualify.

## 8. Resulting Task 4 flow

The overall existing architecture remains:

**</> Plain text**

The important separation is:

**Timer = existing GameEngine responsibility**

**Leaderboard persistence = game/leaderboard.py responsibility**

That's a nice, clean boundary for explaining the implementation during your viva.

---

## 9. Expected leaderboard.json

After the first completed run, you should see a new file in the project root:

</> Plain text

```
12_maze_runner/  
├─ game/  
│   ├── __init__.py  
│   ├── game_engine.py  
│   ├── leaderboard.py  
│   ├── maze.py  
│   └─ player.py  
├─ leaderboard.json  
├─ main.py  
└─ requirements.txt
```

The JSON will look like:

</> JSON

```
[  
  5.21,  
  6.83,  
  7.92,  
  8.41,  
  9.17  
]
```

You don't need to create this file manually. It should be generated when the first successful run is recorded.

---

## 10. Syntax checks

Before running the game:

```
</> powershell  
  
python -m py_compile game\leaderboard.py
```

Then:

```
</> powershell  
  
python -m py_compile game\game_engine.py
```

Then optionally compile everything:

```
</> powershell  
  
python -m compileall game
```

And check whitespace:

```
</> powershell  
  
git diff --check
```

Finally:

```
</> powershell  
  
git status
```

**Do not commit.**

---

## 11. Manual testing

### Test A — Fresh installation state

Before starting, if `leaderboard.json` doesn't exist:

`</>` powershell

Test-Path leaderboard.json

It should normally report:

`</>` Plain text

False

Start the game.

**Expected:** game launches normally without any leaderboard file.

---

## Test B — Complete a run

Solve the maze.

Verify:

- Existing timer works.
  - Win screen appears.
  - Your completion time appears.
  - `leaderboard.json` is created.
  - Your time is present in the JSON.
  - Top-5 leaderboard appears.
- 

## Test C — Fastest score ordering

Complete several runs.

For example, if you somehow produce:

`</>` Plain text

12.5

8.2

10.1

```
6.7  
9.3
```

the leaderboard should become:

```
</> Plain text
```

```
6.7  
8.2  
9.3  
10.1  
12.5
```

---

## Test D — More than five scores

Complete six or more runs.

**Expected:** only five entries remain.

The slowest score should disappear when a faster sixth score enters the top five.

---

## Test E — Current-run highlighting

Complete a run that enters the top five.

**Expected:** that entry is highlighted differently on the win screen.

Then complete a slow run that doesn't enter the top five.

**Expected:** existing top-five entries remain; the slow run is not displayed as a ranked entry.

---

## Test F — Persistence

1. Complete a run.
2. Close the game.
3. Start:

```
</> powershell
```

```
python main.py
```

#### 4. Complete another run.

**Expected:** the previous scores are still present.

This proves that the leaderboard isn't merely stored in memory.

---

## Test G — R behavior

During an unfinished run, press:

```
</> Plain text
```

```
R
```

**Expected:**

- New maze.
  - Player reset.
  - Timer starts again from approximately `0.0s`.
  - No leaderboard entry is created.
  - BFS hint state resets as before.
  - Fog resets as before.
- 

## Test H — H behavior

During gameplay:

```
</> Plain text
```

```
H → H
```

**Expected:** BFS hint still toggles normally.

Verify that using H doesn't alter the timer or leaderboard.

---

## Test I — Corrupted JSON

After creating a leaderboard, close the game and replace `leaderboard.json` contents with something invalid, for example:

**</> Plain text**

```
this is not valid JSON
```

Start the game.

**Expected:**

- Game starts normally.
- No crash.
- Leaderboard behaves as empty.

Then complete a run.

**Expected:** a valid new `leaderboard.json` is written.

---

## Test J — Invalid JSON structure

Try:

**</> JSON**

```
{  
  "score": 10  
}
```

Start the game.

**Expected:** treated as an empty leaderboard; game doesn't crash.

Then try:

**</> JSON**

```
[  
  5.2,  
  "hello",  
  -10,  
  null,  
]
```

```
8.4  
]
```

**Expected:** only valid non-negative numeric scores are retained.

---

## Important: don't commit yet

For Task 4, your intended final Git change should be approximately:

</> Plain text

```
new file:   game/leaderboard.py  
modified:   game/game_engine.py  
new file:   leaderboard.json  ← only after a completed test run
```

**And nothing else.**

Don't create:

</> Plain text

```
feat: add timer leaderboard with JSON persistence
```

yet.

Run the syntax checks and manual tests first. **Task 5 stays completely untouched until you confirm Task 4 works and commit it yourself.**

Task 5 — Add Difficulty Tiers and Main Menu

Task 4 has been completed and committed successfully.

Commit 4:

76cb749 feat: add timer leaderboard with JSON persistence

The working tree is clean.

The project currently has:

1. Task 1 — Correct maze wall collision

2. Task 2 — BFS shortest-path hint
3. Task 3 — Fog of War
4. Task 4 — Timer + persistent top-5 leaderboard using JSON

Now implement ONLY Task 5: Difficulty Tiers / Main Menu.

Do not implement documentation or any final submission work yet.

#### IMPORTANT:

Before changing anything, inspect the ENTIRE current codebase and understand the existing implementation of:

- maze generation
- player movement and wall collision
- BFS hint
- Fog of War
- timer
- leaderboard persistence
- win screen
- reset behavior

Preserve all existing functionality unless a change is genuinely required for difficulty support.

=====

=====

#### TASK 5 REQUIREMENTS

=====

=====

1. Add a proper main menu/state system.

Use a simple state machine such as:

MENU

PLAYING

WON

The game should start in MENU instead of immediately generating a maze.

2. The MENU must provide three clickable difficulty buttons:

Easy

Medium

Hard

The buttons must:

- be clickable with the mouse
- have a visible hover state
- clearly display their difficulty name
- start the game when clicked

3. Difficulty sizes:

Easy:

10 columns × 8 rows

Medium:

15 columns × 13 rows

Hard:

20 columns × 18 rows

4. The selected difficulty must determine:

- maze dimensions
- window dimensions
- exit position
- maze rendering
- player boundaries
- Fog of War surface dimensions
- BFS goal cell
- all other size-dependent calculations

Do NOT leave hard-coded 15 × 13 assumptions in places where they

would break Easy or Hard mode.

5. The current difficulty should be stored as game state.

For example:

```
self.difficulty  
self.cols  
self.rows
```

Use the selected values throughout the game instead of relying on global fixed COLS/ROWS wherever appropriate.

6. Window resizing:

When a difficulty is selected, resize the Pygame window to match:

```
WIDTH = cols * CELL  
HEIGHT = rows * CELL + 60
```

The 60-pixel HUD should remain.

7. R key behavior:

While PLAYING:

R must regenerate a new maze using the CURRENT difficulty.

It must:

- create a new maze
- reset the player
- reset the timer
- clear the BFS hint
- reset Fog of War
- keep the same difficulty

R must NOT return to the menu.

## 8. Escape behavior:

While PLAYING:

Pressing ESC must return to the main menu.

When returning to MENU:

- stop the current game
- do not accidentally record a leaderboard score
- allow another difficulty to be selected

## 9. Win behavior:

When the player reaches the exit:

PLAYING → WON

The existing solved screen should remain functional.

The player should be able to:

- press R to restart the same difficulty
- press ESC to return to the menu

Do not create a new leaderboard entry unless the player actually reaches the exit.

## 10. BFS compatibility:

The existing H hint must continue working at:

Easy

Medium

Hard

The BFS goal must dynamically use:

(rows - 1, cols - 1)

Do not hard-code:

(ROWS - 1, COLS - 1)

if that prevents dynamic dimensions.

11. Fog of War compatibility:

Fog must continue working at all three difficulty sizes.

The fog surface must resize appropriately for the selected maze.

The visibility radius should remain based on CELL rather than the total maze size.

12. Player collision compatibility:

The Task 1 wall collision fix must continue working at all difficulty sizes.

The player must not be able to move through walls or outside the maze.

13. Timer compatibility:

The timer must reset when:

- a new difficulty is selected
- R generates a new maze

It must stop when the player wins.

Returning to MENU with ESC must not create a completed run.

14. Leaderboard compatibility:

The existing JSON leaderboard system must continue working.

Do not break the current persistence behavior.

If the current implementation supports only one global leaderboard, preserve it unless a small change is required by the existing project requirements.

Do not unnecessarily redesign the leaderboard.

#### 15. HUD:

The HUD should continue showing the timer and controls.

Make sure the controls still make sense with the new menu/state system.

For example, while PLAYING it should clearly indicate:

R = New Maze

H = Hint

ESC = Menu

#### 16. MENU appearance:

Create a simple clean menu.

It does NOT need to be visually complicated.

It should clearly show something like:

MAZE RUNNER

Choose Difficulty

[ EASY ]

[ MEDIUM ]

[ HARD ]

Keep the existing visual style where practical.

17. Avoid unrelated refactoring.

Do NOT:

- rewrite the BFS algorithm unnecessarily
- rewrite the maze generator unnecessarily
- rewrite the player collision unnecessarily
- remove the leaderboard
- remove Fog of War
- add new gameplay features
- add sound/music
- add animations
- add difficulty features beyond the requirements

18. Keep the project structure simple.

Only create new files if genuinely necessary.

```
=====
=====
IMPORTANT ARCHITECTURE REQUIREMENT
=====
=====
```

Make sure the existing game functionality is not tightly coupled to:

COLS = 15

ROWS = 13

The selected difficulty must flow through the game correctly.

A good architecture is:

MENU

↓

```

select difficulty
↓
set cols/rows
↓
resize window
↓
generate maze
↓
PLAYING
↓
BFS / Fog / Timer / Player
↓
WON
↓
R → same difficulty
ESC → MENU

```

```

=====
=====
BEFORE CODING
=====
=====

```

First briefly explain:

1. Which files need to change.
2. What state-machine design you will use.
3. How dimensions will be passed to the existing systems.
4. How BFS and Fog of War will remain compatible.
5. How R and ESC will behave.

Then implement the changes.

Do not stop after explaining. Actually implement Task 5.

```

=====
=====

```

## AFTER IMPLEMENTATION

=====

=====

Provide a concise testing checklist covering:

1. Game starts in MENU.
2. Easy button works.
3. Medium button works.
4. Hard button works.
5. Mouse hover works.
6. Window resizes correctly.
7. Easy maze is  $10 \times 8$ .
8. Medium maze is  $15 \times 13$ .
9. Hard maze is  $20 \times 18$ .
10. Wall collision works in all difficulties.
11. H works in all difficulties.
12. BFS path reaches the correct exit.
13. Fog works in all difficulties.
14. Timer works in all difficulties.
15. R regenerates the current difficulty.
16. ESC returns to MENU.
17. Winning changes to WON state.
18. Leaderboard still persists.
19. ESC from PLAYING does not create a leaderboard score.
20. No crashes or regressions.

Do NOT create the Git commit.

I will run the tests first and review the diff before making Commit 5.

Worked for 8s

I can implement Task 5, but there is one important blocker before I safely modify anything.

Your **Task 4 commit 76cb749** is only in your local repository right now. I checked the GitHub repository, and that commit SHA is not available there. So I cannot inspect the actual post-Task-4 code from GitHub without risking that I work from an older version.

Because you explicitly asked me to **inspect the entire current codebase before changing anything**, I don't want to guess at the current `game_engine.py`, especially since Task 5 needs to preserve the exact BFS, fog, timer, and leaderboard integration.

## Please provide the current codebase

The easiest option is to upload the current project as a ZIP, or paste these current files:

</> Plain text

```
main.py
game/game_engine.py
game/maze.py
game/player.py
game/leaderboard.py
```

Also include `leaderboard.json` if it currently exists and you want me to preserve/inspect its format.

**Do not push Commit 5 or make any additional changes yet.**

Once I have those exact post-Task-4 files, I'll:

1. Inspect the complete current implementation.
2. Map all existing hard-coded `15 × 13` assumptions.
3. Design the `MENU → PLAYING → WON` state machine.
4. Implement Easy `10×8`, Medium `15×13`, Hard `20×18`.
5. Make BFS, Fog of War, collision, timer, exit, and leaderboard use the selected dimensions.
6. Keep `R` = restart current difficulty and `ESC` = menu.
7. Leave all unrelated code untouched.
8. Give you the exact syntax checks and full Task-5 test matrix.
9. **Not create or push the Git commit.**

That inspection step is particularly important here—the old `COLS`, `ROWS = 15, 13` constants can affect more than just maze generation, so I want to catch every dependency before changing the architecture.

Today 3:51 PM

## Final Verification — Lab 4 VibeCoding Completion

The Lab 4 Maze Runner project is now complete and all implementation tasks have been completed and committed.

## Final implemented tasks:

### 1. Task 1 — Fixed maze wall collision

Commit:

7bad6f5 fix: correct maze wall collision

### 2. Task 2 — Added BFS shortest-path hint

Commit:

598ec48 feat: add BFS shortest path hint

### 3. Task 3 — Added Fog of War

Commit:

4d67f0d feat: add fog of war

### 4. Task 4 — Added timer and JSON leaderboard persistence

Commit:

76cb749 feat: add timer leaderboard with JSON persistence

### 5. Task 5 — Added difficulty tiers

Commit:

0d8c159 feat: add difficulty tiers

### 6. Added final BEFORE and AFTER demonstration videos

Commit:

5931124 docs: add Lab-4 before and after videos

The project has also been pushed to the GitHub repository:

[https://github.com/rohancodexp/12\\_maze\\_runner](https://github.com/rohancodexp/12_maze_runner)

The BEFORE and AFTER recordings have been completed.

The final implementation should now contain:

- Correct internal maze wall collision
- BFS shortest-path hint using the existing maze wall representation
- H-key hint toggle
- Fog of War around the player
- Timer and top-5 leaderboard persistence

- Easy, Medium, and Hard difficulty modes
- Menu/state handling for difficulty selection
- R reset behavior
- Esc return-to-menu behavior
- Working maze generation and gameplay across difficulty sizes
- BEFORE and AFTER demonstration videos

IMPORTANT:

Do NOT modify, refactor, or add any project features at this stage.

Do NOT generate additional implementation code.

Do NOT implement any future tasks.

Your job now is ONLY to perform a final verification/review of the completed Lab 4 work.

Please provide:

1. A concise final checklist showing each Lab 4 task and whether it is complete.
2. A verification of the Git commit history and explain whether the commits correctly represent the separate tasks.
3. A verification that the BEFORE and AFTER videos are accounted for.
4. A final assessment of whether the project is ready for Lab 4 submission.
5. List ONLY any remaining submission/documentation actions that still need to be done, such as:
  - final GitHub verification
  - adding repository/video links to a document
  - exporting this ChatGPT conversation as PDF
  - preparing the final submission package

Do not suggest further code changes unless there is a genuine

blocking issue.

End with a clear statement:  
"Lab 4 implementation is complete; only final submission/documentation steps remain."

Worked for 15s

## Lab 4 — Final Verification

I verified the GitHub repository and the individual commits you listed. The implementation sequence is coherent, and the BEFORE/AFTER video commit is present in the repository.

### 1. Final task checklist

| Task                              | Status     | Commit  |
|-----------------------------------|------------|---------|
| Task 1 — Wall collision           | ✔ Complete | 7bad6f5 |
| Task 2 — BFS shortest-path hint   | ✔ Complete | 598ec48 |
| Task 3 — Fog of War               | ✔ Complete | 4d67f0d |
| Task 4 — Timer + JSON leaderboard | ✔ Complete | 76cb749 |
| Task 5 — Difficulty tiers/menu    | ✔ Complete | 0d8c159 |
| BEFORE/AFTER videos               | ✔ Added    | 5931124 |

The repository is available at [rohancodexp/12\\_maze\\_runner](#).

### 2. Git commit history verification

The feature commits correctly represent the five separate implementation stages.

#### Task 1

</> Plain text

7bad6f5 fix: correct maze wall collision

Verified: modifies only `game/player.py` .

## Task 2

⟨/⟩ Plain text

598ec48 feat: add BFS shortest path hint

Verified: follows Task 1 and modifies:

⟨/⟩ Plain text

game/game\_engine.py  
game/maze.py

The BFS implementation is based on the existing maze representation.

## Task 3

⟨/⟩ Plain text

4d67f0d feat: add fog of war

Verified: follows Task 2 and modifies `game/game_engine.py` .

## Task 4

⟨/⟩ Plain text

76cb749 feat: add timer leaderboard with JSON persistence

Verified: follows Task 3 and adds/modifies:

⟨/⟩ Plain text

game/game\_engine.py  
game/leaderboard.py

## Task 5

</> Plain text

0d8c159 feat: add difficulty tiers

Verified: follows Task 4 and implements the difficulty/menu work.

The Task 5 commit is substantially larger because it necessarily integrates the existing game systems with the new dynamic maze dimensions/state machine. That is appropriate for this feature and does **not** mean Tasks 1–4 were combined.

## Video commit

</> Plain text

5931124 docs: add Lab-4 before and after videos

This follows Task 5 and is separate from the implementation commits.

There is also a later repository housekeeping commit:

</> Plain text

664210b chore: fix runtime data ignore rules

This is **not a feature implementation commit**. It adjusts `.gitignore` for runtime data ( `leaderboard.json` ), so it does not undermine the requirement that the five implementation tasks have separate commits.

The resulting history therefore has a clean conceptual structure:

</> Plain text

```
Task 1
↓
Task 2
↓
Task 3
↓
Task 4
↓
Task 5
↓
BEFORE/AFTER videos
```



```
runtime-data .gitignore housekeeping
```

That's a perfectly defensible Git history for the assignment.

---

### 3. BEFORE and AFTER videos

This is **verified**, rather than just assumed from the commit message.

Commit:

</> Plain text

```
5931124 docs: add Lab-4 before and after videos
```

contains:

</> Plain text

```
video/before.mp4  
video/after.mp4
```

So both required demonstration recordings are actually accounted for in the GitHub repository.

This matches the Lab-4 handout requirement to record a video before making changes and another after completing the changes.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

---

### 4. Submission readiness

#### Implementation

✓ READY

All five requested implementation stages have been completed and have their own feature commits.

#### Git

✓ READY

The feature work is separated logically into individual commits, as required by the lab.

## GitHub

✓ READY

The repository has been pushed and the feature commits/videos are present remotely.

## Demonstration videos

✓ READY

Both:

⟨/⟩ Plain text

before.mp4

after.mp4

are present in the repository.

## Code scope

✓ READY

There is no apparent need for additional feature work. In particular, I would **not make any more code changes now**. You've reached the point where changing things risks creating unnecessary last-minute problems.

---

## 5. Remaining submission/documentation actions only

Based on the Lab-4 handout, the remaining work is administrative rather than implementation. The handout asks for the videos, updated code, and exported chat history.  Lab\_4\_VibeCoding\_Student\_handout

### Final checklist

- ☐ **Verify the GitHub repository one final time**
  - Confirm `main` is pushed.
  - Confirm the latest expected commits are visible.
  - Confirm `video/before.mp4` and `video/after.mp4` are accessible.
- ☐ **Export this complete ChatGPT conversation as PDF**
  - This conversation documents the prompting and iterative Vibe Coding process.
  - Keep the full conversation because it demonstrates the task-by-task workflow.
- ☐ **Prepare the final Lab-4 submission package**

- Updated code/repository
- BEFORE video
- AFTER video
- ChatGPT conversation PDF

☐ If your submission portal/document asks for links, add the GitHub repository link and any required video links.

☐ Do not make further code changes unless your instructor specifically asks for one.

## Final verdict

Lab 4 implementation is complete; only final submission/documentation steps remain.